

Федеральное государственное автономное образовательное учреждение
высшего образования
«Национальный исследовательский университет ИТМО»

Информационная безопасность
Работа 3: Аудит безопасности веб-приложения

Выполнил:

Баянов Равиль Динарович

Группа:

P3434

г. Санкт-Петербург

2025

Содержание

ХОД РАБОТЫ.....	3
ВЫВОД	15

Ход работы

Воспроизведение уязвимостей

При анализе приложения Juice Shop мы получили такие оповещения:

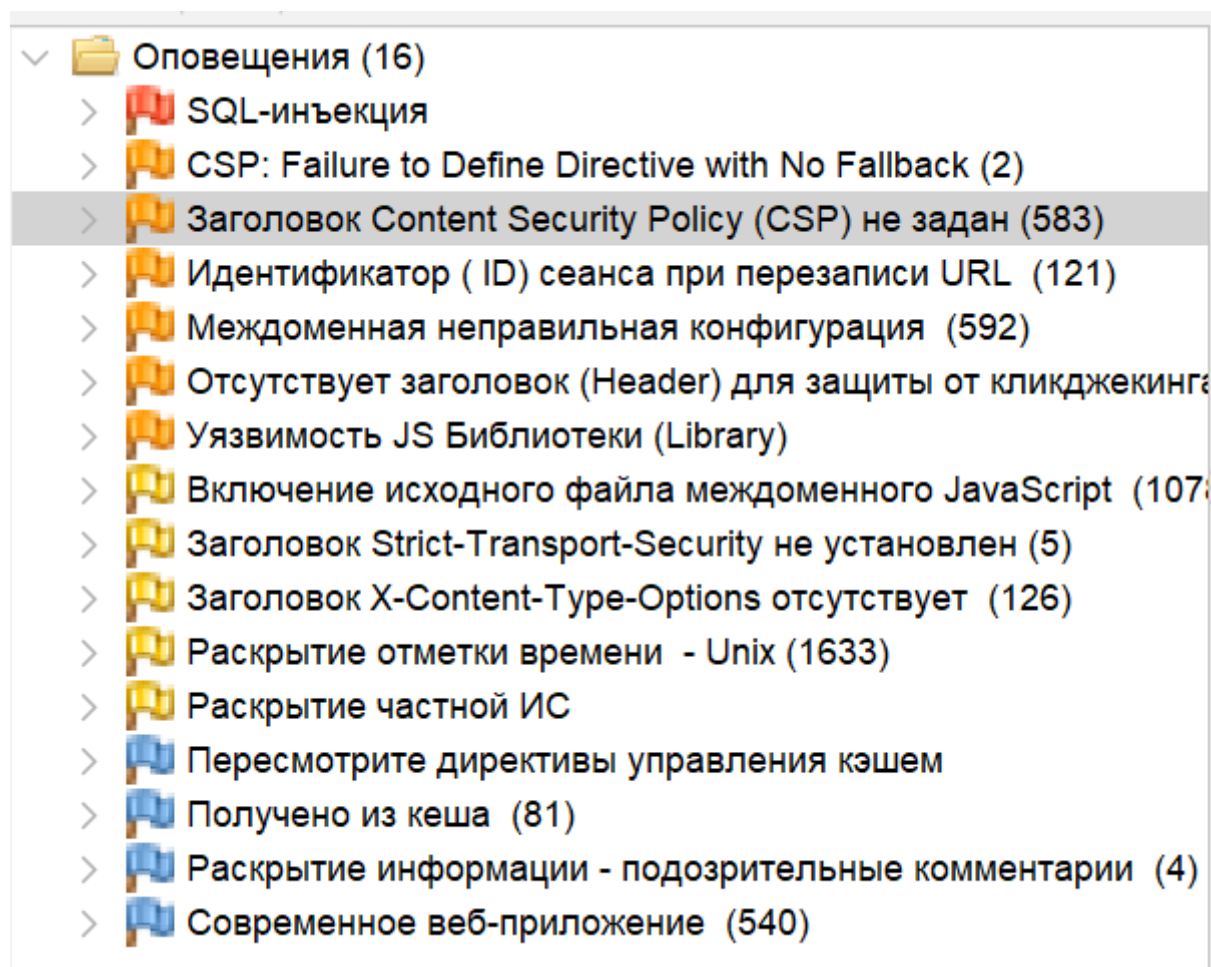


Рисунок 1. Оповещения из ZAP

Проверим достоверность SQL-инъекции:

Введём запрос `test' UNION SELECT * FROM Users--` в поле аутентификации.

Login

Email*

test' UNION SELECT * FROM Users--

Password*

.....



Forgot your password?



Log in

☐

Remember me

or



Log in with
Google

Not yet a customer?

Рисунок 2. Окно аутентификации в приложении

Как видим у меня получилось достать логины и пароли с тестовой БД и зайти на сайт:

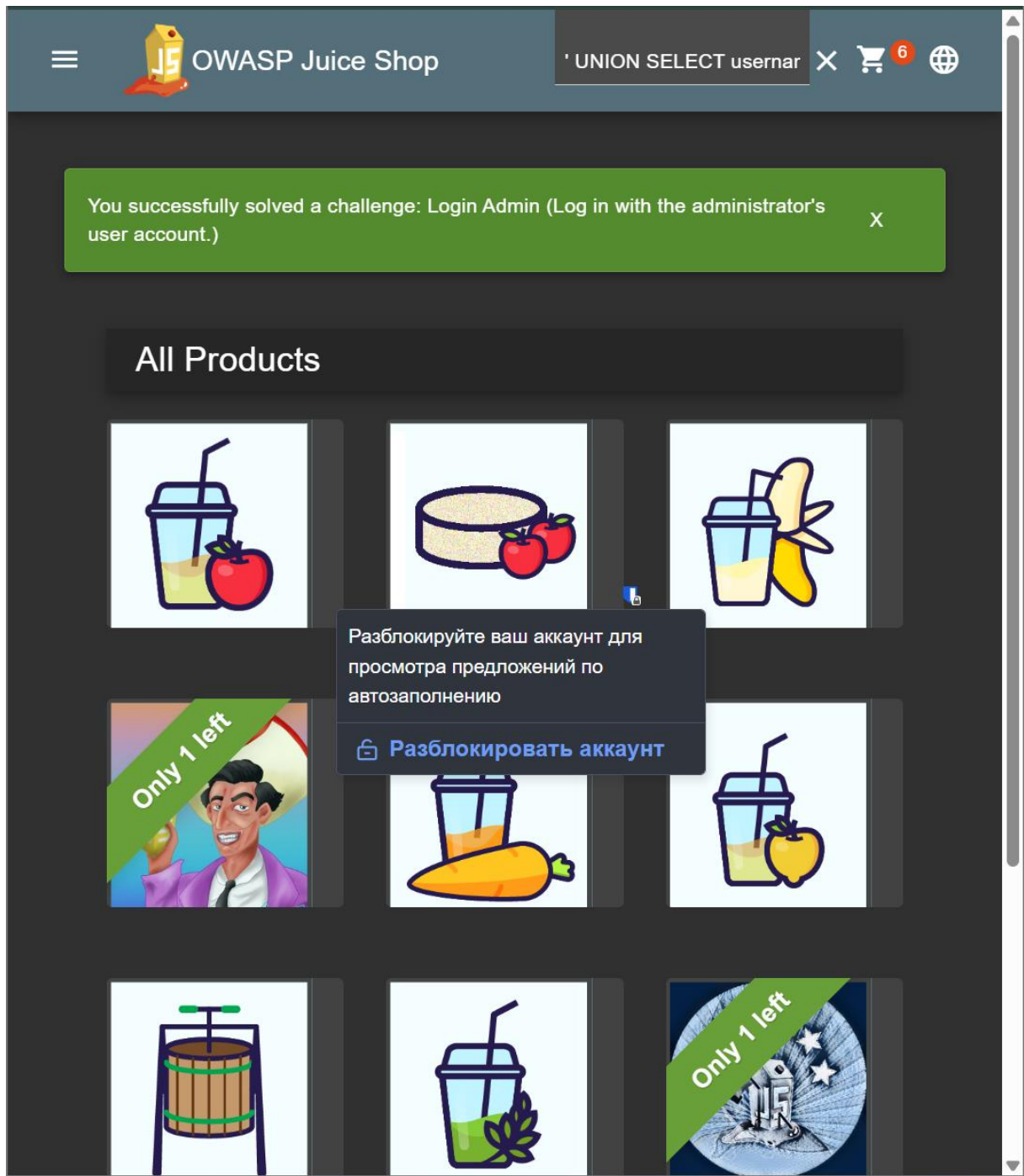


Рисунок 3. Воспроизведение собственной SQL-инъекции

Проверим также XSS, введём в поле поиска продуктов запрос `<img src=x onerror=alert(1)>` и получим окно alert на весь экран:

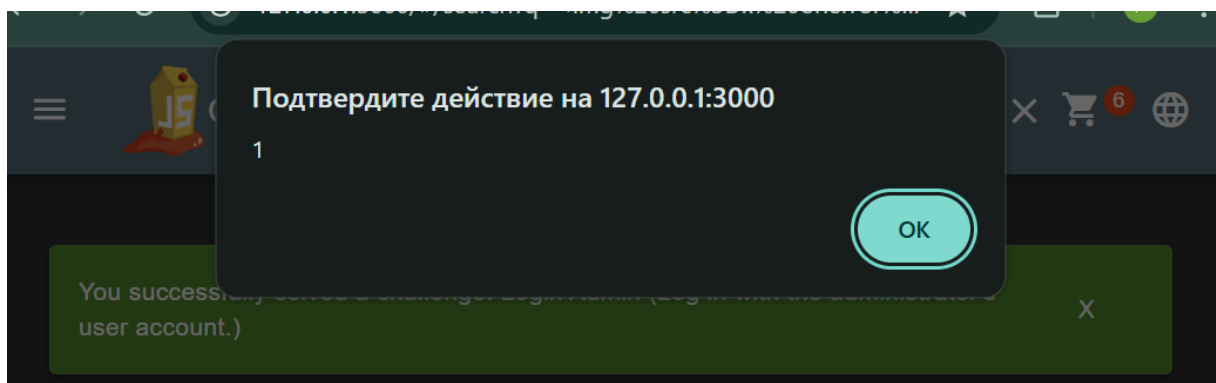


Рисунок 4. Воспроизведение XSS уязвимости

Заметим также, что при вводе запроса ‘OR 1=1 – также можно попасть на сайт.

Проверим также SQL-инъекцию, найденную ZAP, а именно:



Рисунок 5. Окно оповещения о SQL-инъекции в ZAP

И получим ошибку 500:

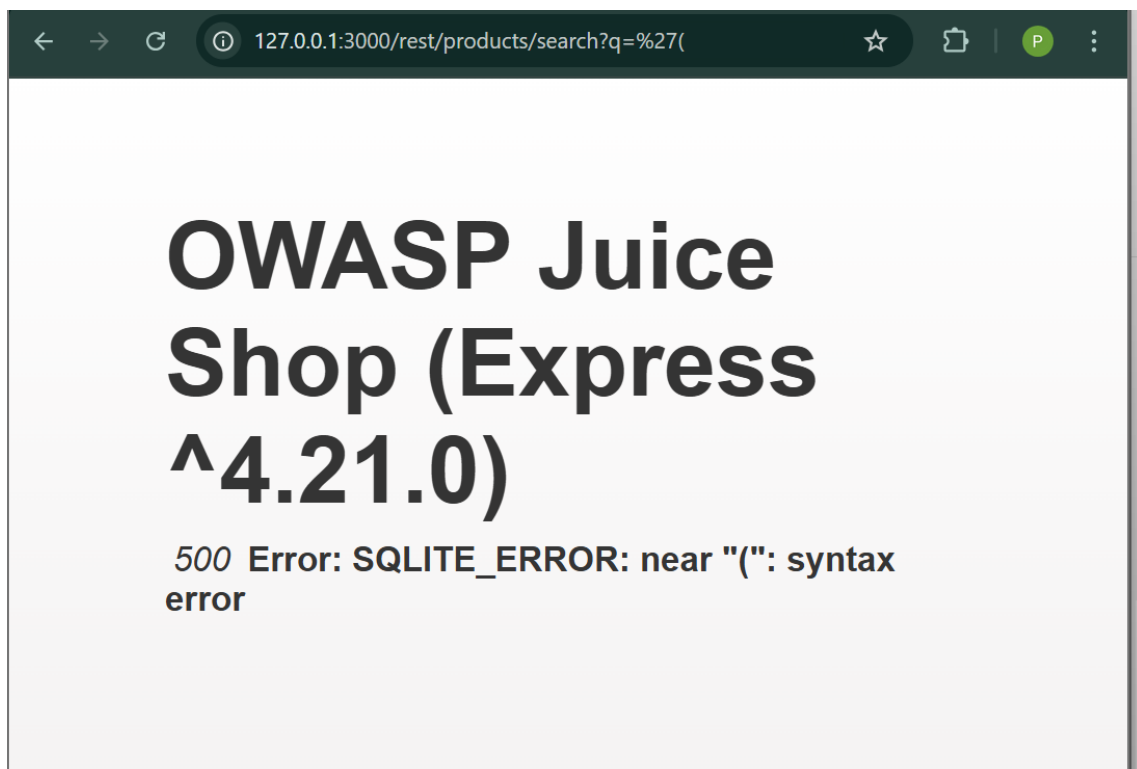


Рисунок 6. Воспроизведение SQL-инъекции из ZAP

Также заметим, что после авторизации в приложении JWT токен сохраняется в localStorage:

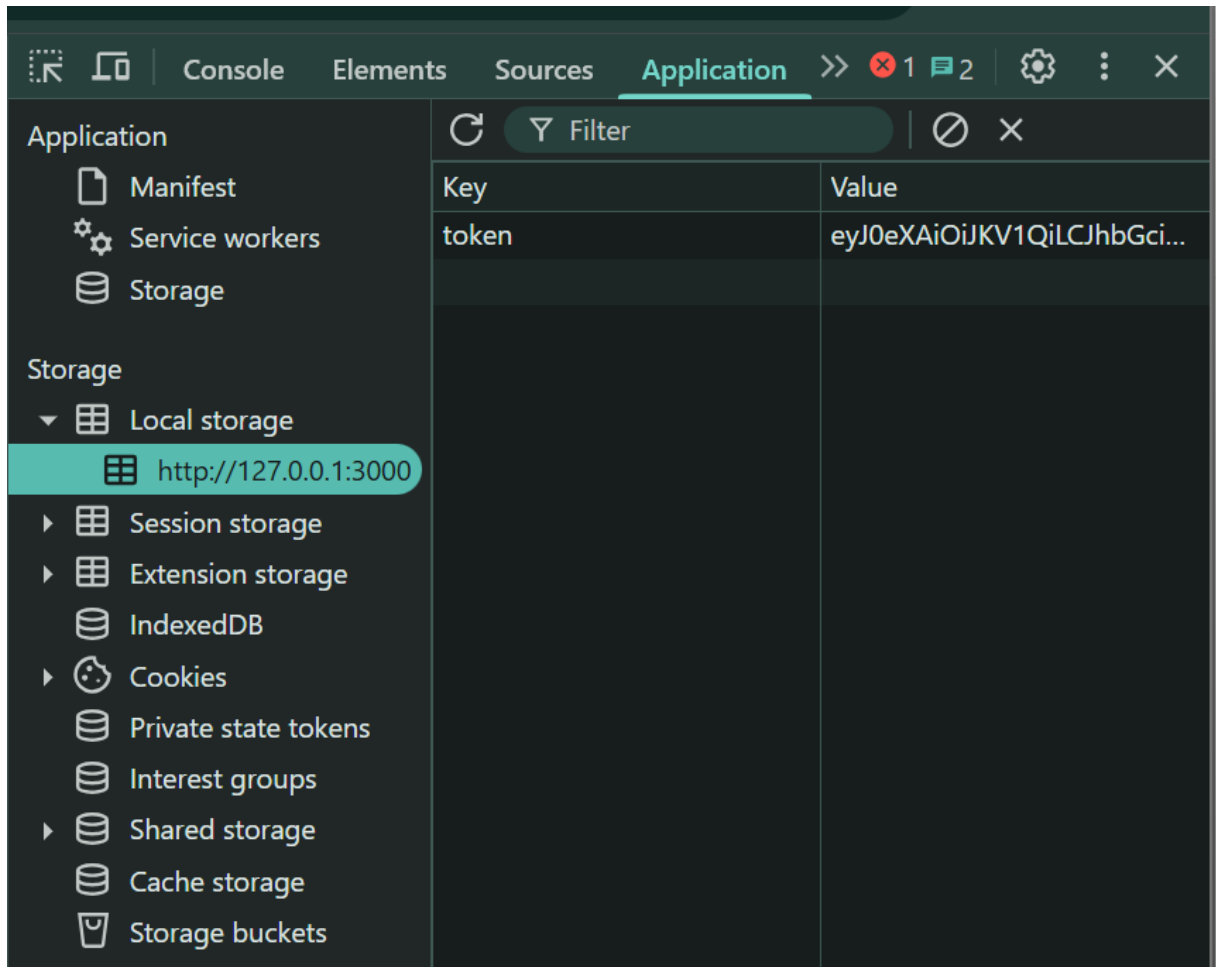


Рисунок 7. Подтверждение наличие токена в localStorage

Увидим, что с помощью него можно зайти на сайт.

Моделирование угроз (Threat Modeling) с помощью STRIDE

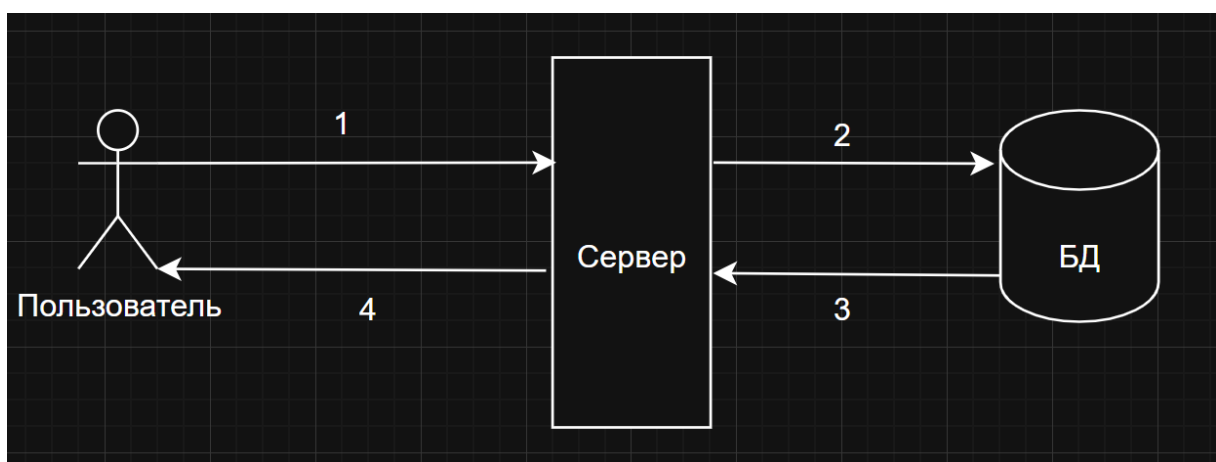


Рисунок 8. Модель системы Juice Shop

Теперь распишем потоки для данной системы, каждая цифра будет соответствовать части одного потока:

- Аутентификация пользователя
 1. Отправка логина и пароля на веб-сервер
 2. Запрос данных пользователя из базы данных
 3. Возвращение хэша пароля серверу
 4. Возвращение токена пользователю
- Поиск товаров
 1. Отправляем поисковой запрос на сервер
 2. Отправляем запрос в базу данных
 3. Отправляем полученные данные на сервер
 4. Возвращаем пользователю ответ
- Отправка отзывов
 1. Отправляем отзыв на сервер
 2. Сохраняем отзыв в базе данных
 3. Отправляем серверу список обновленных отзывов
 4. Возвращаем пользователю обновленный список отзывов
- Оформление заказа
 1. Отправляем выбранные товары на сервер
 2. Обновляем данные в базе данных
 3. Возвращаем результат оформления заказа
 4. Возвращаем пользователю статус о заказе

Анализ угроз по STRIDE

- Spoofing (Подмена личности)
 - Злоумышленник использует похищенный токен и выполняет вход от имени пользователя.
- Tampering (Искажение/подмена данных)
 - Изменение содержимого заказа в пути – клиент отправил товары, а посредник или уязвимый сервер поменял цену.
- Repudiation (Отказ от действия)
 - Пользователь утверждает, что не отправлял отзыв, а в системе нет аудитов для отслеживания этого.
- Information Disclosure (Утечка информации)
 - Похищение данных с помощью, например, SQL-инъекции.
- Denial of Service (Отказ в обслуживании)
 - Атака массовым количеством запросов, чтобы перегрузить систему
- Elevation of Privilege (Повышение привилегий)
 - Злоумышленник через нарушения в логике API получает возможность стать админом, что даёт, например, изменить статус заказа.

Таблица уязвимостей

Название	Описание	Уровень риска (CVSS)	Категория OWASP Top 10	Предложение по исправлению
SQL-инъекция в поле аутентификации ' OR 1=1 или же test' UNION SELECT * FROM Users-	Ввод пользователя напрямую подставляется в SQL-запрос логина/пароля — злоумышленник может вводить payload'ы типа ' OR 1=1 или test' UNION SELECT * FROM Users--, чтобы обойти аутентификацию или получить данные пользователей.	9.8 — Critical	A03: Injection	Использовать подготовленные выражения (parameterized queries), ORM с безопасной подстановкой, не строить SQL конкатенацией; минимальные права у БД; не показывать отладочные SQL-ошибки; внедрить rate-limit/lockout и мониторинг попыток входа; тестирование (DAST/SAST).
Reflected XSS в поле поиска	Ввод отражается в ответе сервера без экранирования → выполнение произвольного JS в браузере жертвы (угроза кражи сессий, подмена содержимого, CSRF-усиления).	7.5 — High	A03: Injection	Контекстно-осознанное экранирование вывода; не вставлять пользовательский ввод в HTML как HTML; использовать textContent/шаблонизаторы с автоэскейпинг; CSP; избегать хранения токенов в доступном JS

				(localStorage); санитизация (DOMPurify) только если нужен HTML; WAF/DAST + мониторинг.
SQL- инъекция в пользователь- ском вводе	При вводе строки '(в параметр (например, поле поиска или аутентификации) приложение генерирует SQL- запрос с некорректным синтаксисом, что приводит к ошибке 500. Это указывает на отсутствие корректной обработки ввода пользователя и подтверждает уязвимость к SQL- инъекциям. Подобный дефект позволяет злоумышленнику тестировать структуру базы данных, строить вредоносные запросы (например, UNION SELECT) и потенциально получать доступ к конфиденциальным данным.	9.1 - Critical	A03:2021 — Injection	Заменить конкатенацию SQL на подготовленные выражения (Prepared Statements) / parameterized queries. Никогда не подставлять пользовательски й ввод напрямую в текст запроса.
Кража JWT токена через localStorage	JWT (access token) хранится в localStorage — при наличии XSS злоумышленник может прочитать токен и совершать действия от имени	8.0 — High	A07:2021 — Identification and Authentication Failures	Не хранить чувствительные токены в localStorage. Использовать HttpOnly; Secure; SameSite cookies для

	пользователя.			access/refresh токенов, минимальное время жизни access token, refresh токены с защитой и проверкой привязки к клиенту; устранять XSS; MFA для критичных операций; возможность инвалидации сессий/токенов.
Похищенный токен — вход от имени пользователя	Злоумышленник использует украденный JWT/сессионный токен и получает доступ в систему под чужим аккаунтом, совершая действия от имени владельца (поиск, отзывы, оформление заказов)	8.5 — High	A07:2021 — Identification and Authentication Failures	Инвалидировать /ротировать токены, хранить токены в HttpOnly; Secure; SameSite cookie, минимальный TTL для access token, рефреш с ротацией и проверкой привязки, MFA для чувствительных операций, мониторинг аномалий, возможность немедленной инвалидации (blacklist).
Изменение содержимого заказа в пути (man-in-the-middle / серверная подмена	Клиент отправил список товаров с ценами, но посредник или уязвимый сервер изменил цены/артикулы/количество в пути или при	8.8 — High	A08:2021 — Software and Data Integrity Failures	Сервер — единственный источник истины для цен: вычислять итог на сервере по доверенной

цены)	обработке → пользователь платит другую сумму или оформляется неверный заказ.			прайс-таблице; подписывать/проверять целостность корзины (HMAC) при необходимости; TLS; транзакционная обработка; аудит и оповещения о изменениях; валидация на сервере всех критичных полей.
Отсутствие аудита при изменении/добавлении отзывов	Пользователь утверждает, что не отправлял отзыв, но система не ведёт журнал действий. Нет возможности отследить, кто и когда создал или изменил отзыв. Это позволяет злоумышленнику фальсифицировать отзывы или скрыть следы действий.	7.5 — High	A08:2021 — Software and Data Integrity Failures	Внедрить аудит действий пользователей: логирование создания, редактирования и удаления отзывов с указанием пользователя, времени, IP/UA; обеспечить неизменяемость логов (immutable logs); возможность восстановления истории действий; при необходимости — хранить контрольные суммы или подписи записей.
Похищение данных через SQL-инъекцию	Через уязвимость SQL-инъекции злоумышленник выполняет произвольные	9.8 — Critical	A03:2021 — Injection	Устранить инъекции (parameterized queries/ORM), ограничить

	SELECT/UNION/EXT RACT-запросы и получает конфиденциальные данные (ПІ, пароли, платёжные данные и т.д.)			права БД (least privilege), шифрование данных в покое и в транзите, мониторинг и DLP-контроль, WAF, аудит запросов и пентесты.
Атака массовыми запросами (DoS / DDoS)	Злоумышленник генерирует огромное количество запросов (или специально crafted запросы), чтобы исчерпать ресурсы сервера/БД и сделать сервис недоступным для легитимных пользователей. Может быть прикрыта бот-сетью (DDoS) или реализована через легитимные-looking запросы (application-layer flood).	7.5 — High	A05:2021 — Security Misconfiguration	Внедрить rate limiting, WAF/anti-DDoS, CDN/edge caching, autoscaling + circuit breakers, backpressure и очереди, ресурсные квоты, проверку/капчу для подозрительных потоков, мониторинг и автоматическую блокировку аномалий.
Повышение привилегий через логику API (уязвимость доступа / Broken Access Control)	Из-за ошибок в проверках прав/логике API злоумышленник получает возможность присвоить себе роль администратора или выполнить админские операции (например, изменить статус чужого заказа).	9.0 — Critical	A01:2021 — Broken Access Control	Жёсткие серверные проверки авторизации на каждом эндпоинте; RBAC/ABAC; не доверять клиентским флагам роли; валидация прав в бизнес-логике; least privilege; аудит и тесты на обход прав.

Общие рекомендации по усилению безопасности

SQL-инъекции и XSS

- Использовать подготовленные выражения (prepared statements) для всех запросов к БД.
- Экранировать и санитизировать весь вывод на фронтенде; внедрить строгую CSP.

Токены и сессии

- Хранить JWT и refresh-токены в HttpOnly и Secure cookie, не в localStorage.
- Вводить ротацию токенов и инвалидацию при подозрительной активности.

Авторизация и привилегии

- Централизованный RBAC/ABAC на сервере; не доверять клиентским ролям.
- Для критичных операций (изменение статуса заказа, роли) — step-up authentication или MFA.

Аудит и логирование

- Логировать ключевые действия: login, CRUD отзывов, изменения заказов и ролей.
- Логи immutable, с IP, timestamp и userId для расследования инцидентов.

Защита от перегрузки (DoS) и rate limiting

- Ограничивать количество запросов на ключевых эндпоинтах (login, search, checkout).
- Использовать CDN, WAF и очереди для тяжёлых операций, чтобы снизить нагрузку на сервер.

Вывод

Выполнив данную лабораторную работу, я научился анализировать сайты на безопасность, а также идентифицировать уязвимости. Теперь я осознал, как много аспектов нужно учесть при создании информационных систем, дабы сохранить безопасность данных.